Mikhail Razakov

Quant Developer / Research Software Engineer — C++ · Python

Moscow / remote · razakov.mikhail@outlook.com · Telegram · GitHub

PROFILE

I develop algorithmic tools and reproducible research experiments in C++, GNU C and Python. My work includes exact methods, reference oracles, time-series data processing and counterexample analysis. I built a research package around XAGUSD OHLC data with a deterministic benchmark and blind calibration; these results do not establish trading profitability. I work on symbolic computation at ISP RAS and previously completed a C++/LLVM compiler internship at MCST.

TECHNICAL SKILLS

Languages and data: C++23, GNU C17, Python, NumPy, pandas, SciPy, SQL; LLVM 22, Git, Linux, CMake

Algorithms: Data structures, graph algorithms, memory dependence, integer arithmetic, affine models, OHLC/time-series processing

Research engineering: Independent oracles, randomized/metamorphic and differential testing, deterministic datasets and reproducible counterexamples

EXPERIENCE

ISP RAS · SharpChecker · October 2026 — present

- Working on symbolic computation for C#/.NET static program analysis.

MCST · Compiler Engineering Internship · July — August 2026

- Worked on experimental LLVM LICM and Python-based before/after checks. Implemented RPO, Dijkstra, Dinic and Tarjan SCC in C++23; a local run passed 14/14 fixture tests.

ENGINEERING PROJECTS

PS-form Memory Dependence Analyzer

Built a GNU C17 analyzer with exact counting for supported affine cases, modular/GCD checks and an independent reference oracle. Unresolved cases return conservative maybe. An extended smoke run identified five completeness limitations, not a demonstrated false proof of independence.

Simple Parallel Channel Tracker

Created a private Python OHLC research package (NumPy/pandas/SciPy) with 10 timeframes and a deterministic benchmark generator. Structural checks passed: 31 machine-labeled cases, 6 nested pairs, 3 successor sequences; blind-calibration pack with 20 cases and 8 relations. There is no expert gold set; predictive accuracy and trading returns have not been established. Source remains private.

LLVM Interprocedural Global IV Optimization

C++23/LLVM 22: interprocedural effect and legality analysis with selected before/after execution comparisons. Passed 29/29 regression scenarios on LLVM 22.1.8, including conservative rejection paths.

EDUCATION

HSE University — Software Engineering

RECOGNITION

2026 · MCST LLVM/C++ selection: According to organizer correspondence: 5.0/5.0 and 104/104 official tests among 49 submitted solutions.

2025–2026 · MEPhI Junior: First-degree diplomas in the Information Technologies section in 2025 and 2026.

2026 · LangDev — UniversalToolchain: Delivered a talk on 9 October 2026; the organizer published the recording. — Talk recording